

A. Merkle Tree

What it is

A **Merkle tree** is a binary hash tree built from transaction hashes.

- Leaves = hashes of individual transactions
- Internal node = hash of concatenation of child hashes
- Root = one final hash representing all transactions in the block

Why it is used

- Efficient proof that a transaction belongs to a block
- Only a small path is needed, not the whole block
- Saves storage and bandwidth
- Useful for SPV / lightweight verification

How to build it

Suppose tx hashes are:

- H1
- H2
- H3
- H4

Then:

- $A = \text{hash}(H1 \parallel H2)$
- $B = \text{hash}(H3 \parallel H4)$
- $\text{Root} = \text{hash}(A \parallel B)$

If odd number of leaves, usually the last hash is duplicated.

Example:

- H1, H2, H3
- $A = \text{hash}(H1 \parallel H2)$
- $B = \text{hash}(H3 \parallel H3)$
- $\text{Root} = \text{hash}(A \parallel B)$

Merkle Proof

A Merkle proof shows that a transaction is included in a block.

You need:

- the transaction hash
- sibling hashes along the path
- position info (left/right)

Then recompute upward until root.

If computed root = block's Merkle root, proof is valid.

Common exam traps

- Forgetting order of concatenation
 - Mixing left and right sibling positions
 - Not duplicating last leaf when odd number of transactions
 - Using raw transaction instead of transaction hash
-

B. Bitcoin Transaction Signature Construction and Verification

This is one of the most likely big exam questions.

Big idea

Bitcoin uses digital signatures to prove that the spender is authorized to spend an output.

In simple words

To spend a UTXO:

- transaction is created
- message/digest of relevant transaction data is formed
- owner signs that digest with **private key**
- verifier checks signature with **public key**

Construction idea

Very simplified exam view:

1. Create transaction data
2. Create the message to be signed
3. Hash it

4. Sign hash using private key
5. Put signature in unlocking data / witness depending on type

Verification idea

1. Recreate correct message/digest
2. Hash it
3. Use public key and signature
4. Verify signature mathematically
5. If valid, spender is authorized

What a valid signature proves

- spender has corresponding private key
- transaction data was not altered after signing

What it does NOT prove

- that the spender is a “good person”
- that the input has value left unless UTXO exists
- that the whole transaction is economically sensible

Consequences of invalid signature

- input fails unlocking
 - transaction invalid
 - miners/nodes reject it
-

C. Bitcoin UTXO Unlocking

This is extremely important.

UTXO

UTXO = **Unspent Transaction Output**

A Bitcoin balance is not stored like a bank account balance.
Instead, you own spendable outputs from older transactions.

Spending a UTXO means

You reference an earlier output and provide data that satisfies its locking condition.

General structure

Previous output contains **locking script**

New spending transaction contains **unlocking script** or witness data

To spend successfully:

- unlocking data + locking conditions must evaluate to true
-

P2PKH unlocking

P2PKH = Pay to Public Key Hash

Locking idea

“Only the person who can provide:

- correct public key
- valid signature
for this public key hash
may spend.”

Typically requires

- signature
- public key

Verification logic

1. Hash provided public key
 2. Compare with stored pubkey hash
 3. If match, verify signature
 4. If valid, spend succeeds
-

P2MS unlocking

P2MS = Pay to Multisig

Idea

Requires **m-of-n** signatures.

Example:

- 2-of-3 multisig
- any 2 valid signatures out of 3 public keys

Used for

- shared control
 - treasury
 - joint authorization
-

P2SH unlocking

P2SH = Pay to Script Hash

Idea

Instead of locking to a public key hash, lock to a **redeem script hash**.

To spend:

- provide redeem script
- hash of redeem script must match stored hash
- then execute redeem script
- provide whatever data redeem script needs

Why used

Makes complex scripts easier to send to
Only script hash appears on chain initially

P2WPKH

SegWit version of P2PKH.

- signature and pubkey are placed in **witness**
 - main benefit: lower weight, fixes malleability issues better than legacy style
-

P2WSH

SegWit version of P2SH-like script hashing.

- witness carries script and witness data

- script hash is committed in output
-

P2TR

Taproot output type.

High-level idea

- more privacy
- more efficiency
- can support key-path spend and script-path spend
- modern Bitcoin output type

For exam:

Know it is newer, more efficient, more private/flexible compared to older forms.

D. Proof-of-Work and Mining

Another super likely numerical.

What miners do

Miners try to find a block header hash below a target.

Condition

Valid if:

$\text{hash}(\text{block header}) < \text{target}$

Nonce

A nonce is a number miners keep changing to get a new hash output.

Mining process

1. Build candidate block
2. Include transactions
3. Compute Merkle root
4. Form block header
5. Change nonce

6. Hash header
7. Check if hash < target
8. If yes, block valid

Lower target means

- harder mining
- fewer hashes qualify
- more attempts needed

Common question styles

- Given target and several hash outputs, which one is valid?
- Given several nonce trials, which nonce succeeds?
- Why does miner keep changing nonce?
- What if no nonce works in shown range?

Consequences

If hash is above target:

- block invalid for PoW
 - miner continues searching
-

E. UTXO Database

What it is

A node maintains a database/set of all currently unspent outputs.

Why important

To verify whether an input is spendable.

When transaction is processed

For each input:

- referenced UTXO must exist
- must not already be spent
- unlocking must succeed

Then:

- referenced old outputs are removed from UTXO set
- new outputs created by this transaction are added to UTXO set

In short

Inputs consume old UTXOs

Outputs create new UTXOs

Common exam reasoning

If referenced output is not in UTXO database:

- transaction invalid

If same UTXO used twice:

- double-spend attempt
-

F. Transaction Fee and Fee Rate

Very likely numerical.

Transaction fee

Fee = total input value – total output value

Example

Inputs = 2.5 BTC

Outputs = 2.3 BTC

Fee = 0.2 BTC

Fee rate

Fee rate = fee / transaction size

Usually in:

- satoshis per byte
- satoshis per vbyte

Why fee rate matters

Miners usually prefer transactions with higher fee rate.

Common traps

- students subtract wrong direction
 - confuse total fee with fee rate
 - forget to sum all inputs or all outputs
-

G. Transaction types summary table

Here is the cleanest way to remember them:

Type	Full Form	Spend requires	Main idea
P2PKH	Pay to Public Key Hash	signature + public key	classic standard payment
P2MS	Pay to Multisig	multiple signatures	shared authorization
P2SH	Pay to Script Hash	redeem script + needed data	pay to hash of script
P2WPKH	Pay to Witness Public Key Hash	witness signature + pubkey	SegWit P2PKH
P2WSH	Pay to Witness Script Hash	witness script + data	SegWit script hash
P2TR	Pay to Taproot	taproot key/script path	modern efficient/private

4) Super short conceptual answers you should memorize

What happens if Merkle proof fails?

The computed root will not match the block's Merkle root, so transaction inclusion is not proven.

What happens if signature is invalid?

The input cannot unlock the referenced UTXO, so the transaction is rejected.

What happens if referenced output is not in UTXO database?

The input tries to spend a non-existent or already spent output, so transaction is invalid.

Why is nonce changed?

Changing nonce changes block header hash so miner gets a new chance to satisfy the target.

Why are transaction fees needed?

They incentivize miners and help prioritize which transactions get included.

Why is fee rate more useful than total fee?

Because miners compare profit relative to space consumed in the block.

Why are Merkle trees useful?

They allow efficient proof of inclusion without downloading every transaction.

5) Last-minute solving method for numericals

For any simulation question:

For Merkle tree

- write leaf hashes in order
- combine pair by pair
- duplicate last if odd
- keep order consistent

For signature/unlocking

- identify output type first
- ask: what data is needed to unlock this?

- check whether provided data satisfies the lock

For UTXO questions

- mark old UTXOs consumed
- add new outputs created
- update UTXO set carefully

For fee questions

- sum inputs
- sum outputs
- subtract outputs from inputs

For fee rate

- compute fee first
- divide by size

For PoW

- compare each hash against target carefully
- smaller-than-target wins